



Chapter 3

Dealing with Model Overfitting and Underfitting

IN THIS CHAPTER

» Defining overfitting and underfitting

» Considering model problem sources

» Understanding the role of features

.....

A *model* is the description of the data points in the form of an algorithm (often represented by a mathematical function). [Book 3](#) discusses various kinds of modeling associated with particular data point patterns. For example, data points that form a straight line rely on linear regression. The purpose of creating a model is to either predict the location of future data points or to categorize data based on where it falls within the model. However, a model is only as good as the underlying algorithm. An algorithm that follows the original data points too closely *overfits* the curve to the data. An algorithm that doesn't follow the original data points well enough *underfits* the curve to the data. Of course, overfitting and underfitting are both problems, which is why you need this chapter. Unless a model runs true to the data, anything you use the model for is suspect.

After you know why overfitting and underfitting occur, you need to consider the sources of these two problems. In some cases, the problem is definitely related to the data itself, but often the problem is in the manner that the data is collected and manicured for use. Poor assumptions about the data can also cause problems.

One of the most important concerns with overfitting and underfitting pertains to the *features* you select — that is, one or more columns in the data table containing the data. Each of the rows is a *case* (or sometimes called an item, event, or instance). Analysis involves viewing the features for each case and using them to construct the model. When you choose the wrong features, the model won't work. The last part of this chapter discusses working with features.



REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0603_Fitting.ipynb` source code file for Python and the `DSPD_R_0603_Fitting.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Understanding the Causes

It's an oversimplification to state that overfitting and underfitting are issues surrounding the construction of a model based on the data. Something must cause the problem to occur. In fact, more than one problem can occur with the modeling of data, making the solution to overfitting and underfitting more complex than you might think. The following sections discuss common causes of overfitting and underfitting so that you can better understand how to locate the causes of these problems in your own models.

Considering the problem

Fitting a model implies learning from data a representation of the rules that generated the data in the first place. From a mathematical perspective, fitting a model is analogous to guessing an unknown function of the kind you faced in high school, such as $y=4x^2+2x$, just by observing its y results. Therefore, under the hood, you expect that data analysis algorithms generate mathematical formulations by determining how reality works based on the examples provided.



REMEMBER Demonstrating whether such formulations are real is beyond the scope of data science. What is most important is that they work by producing exact predictions. For example, even though you can describe much of the physical world using mathematical functions, you often can't describe social and economic dynamics this way, but people try guessing them anyway.

To summarize, as a data scientist, you should always strive to approximate the real, unknown functions underlying the problems you face by using the best information available. The result of your work is evaluated based on your capacity to predict specific outcomes (the target outcome) given certain premises (the data) thanks to a useful range of algorithms (the machine learning algorithms).

Earlier in the book, you see something akin to a real function or law when the book presents a linear regression, which has its own formulation. The linear formula $y=Bx + a$, which mathematically represents a line on a plane, can often approximate training data well, even if the data is not representing a line or something similar to a line. As with linear regression, all other algorithms used for tasks such as machine learning have an internal formulation (and some, such as neural networks, even require you to define their formulation from scratch). The linear regression's formulation is one of the simplest ones; formulations from other learning algorithms can appear quite complex. You don't need to know exactly how they work. You do need to have an

idea of how complex they are, whether they represent a line or a curve, and whether they can sense outliers or noisy data. When planning to learn from data, you should address these problematic aspects based on the formulation you intend to use:

1. Whether the learning algorithm is the best one to approximate the unknown function that you imagine is behind the data you use. In order to make such a decision, you must consider the learning algorithm's formulation performance on the data at hand and compare it with other, alternative formulations from other algorithms.
2. Whether the specific formulation of the learning algorithm is too simple, with respect to the hidden function, to make an estimate (this is called a *bias* problem).
3. Whether the specific formulation of the learning algorithm is too complex, with respect to the hidden function you need to guess (leading to the *variance* problem).



REMEMBER Not all algorithms are suitable for every data problem. If you don't have enough data or the data is full of erroneous information, some formulations may have too much difficulty figuring out the real function.

Looking at underfitting

As previously mentioned, underfitting means that the model you create doesn't actually follow the data points very well, so any prediction or categorization is suspect. Of course, using visualization to see how this process works is helpful. The first step is to generate some data using this code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
%matplotlib inline
```

```
np.random.seed(51)
```

```
x = np.array(range(1, 50))
```

```
vary = (np.random.random(len(x))) / 5
```

```
y = np.sin(x * np.pi / 50.) + vary
```

What this code normally generates is a curve based on sine. Without `vary`, the output would be a nearly perfect sinewave, in fact. Adding `vary` means that the data will look more like the real world with little variations in the data points. Setting the `np.random.seed()` value ensures that the random values provided by `vary` remain the same for each test.

You can simulate underfitting by using a linear regression, as discussed in Book 3, [Chapter 1](#). For this example, the degree of polynomial fitting provided by `polyfit()` as described at

<https://docs.scipy.org/doc/numpy/reference/generated/numpy.polyfit.html>

becomes important. A linear regression uses a single degree of polynomial fitting, as shown in the following code:

```
plt.scatter(x, y)
```

```
z = np.polyfit(x, y, 1)
```

```
p = np.poly1d(z)
```

```
print(p)
```

```
plt.plot(x, p(x), 'g-')
```

```
plt.legend(['Model', 'Data'])
```

```
plt.show()
```

[Figure 3-1](#) shows the output of this example. Notice that the line charges straight through the data, so any analysis would be accurate only where the model happens to cross the data, which is only two points in the range.

Examination of `p` shows that the model does indeed use only one degree of fitting:

```
0.0005854 x + 0.7309
```

When your model isn't complex enough, it won't fit your data properly, and you end up with inaccuracy rather than something useful.

Looking at overfitting

Overfitting means that the model fits the data too closely, so the model is unlikely to work with any new data you provide. The data developed in the previous section can also work to demonstrate overfitting. In fact, all you really need to do is add degrees of polynomial fitting, as shown in the following code:

```
plt.scatter(x, y)
```

```
z = np.polyfit(x, y, 18)
```

```
p = np.poly1d(z)
```

```
print(p)
```

```
plt.plot(x, p(x), 'g-')
```

```
plt.legend(['Model', 'Data'])
```

```
plt.show()
```

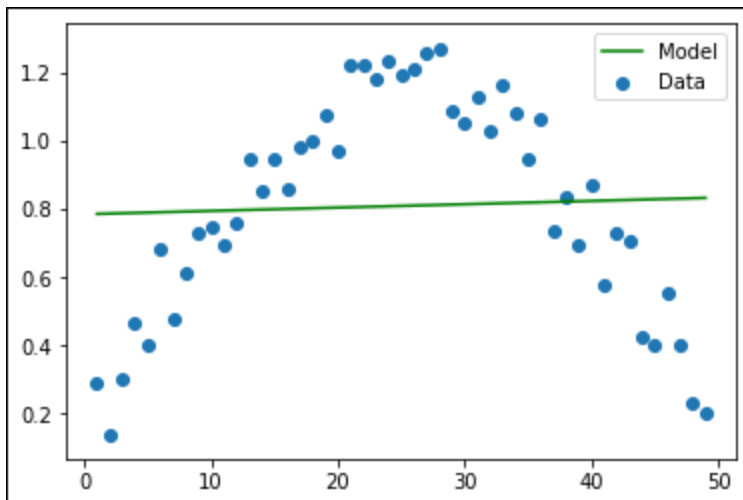


FIGURE 3-1: Underfitting is the result of using a model that isn't complex enough.

In this case, you use 18 degrees of polynomial fitting. Printing `p` demonstrates this fact:

18

17

16

15

14

```
1.44e-22 x-6.054e-20 x+1.17e-17 x-1.379e-15 x+1.109e-13 x
```

13

12

11

10

```
- 6.465e-12 x+2.826e-10 x-9.461e-09 x+2.458e-07 x
```

9

8

7

6

5

```
- 4.99e-06 x+7.923e-05 x-0.0009799 x+0.00935 x-0.06763 x
```

4 3 2

+ 0.3602 x-1.346 x+3.247 x-4.253 x+2.339

The graphic output of the example differs considerably, too. [Figure 3-2](#) shows the results of overfitting against the data for which the model is created.

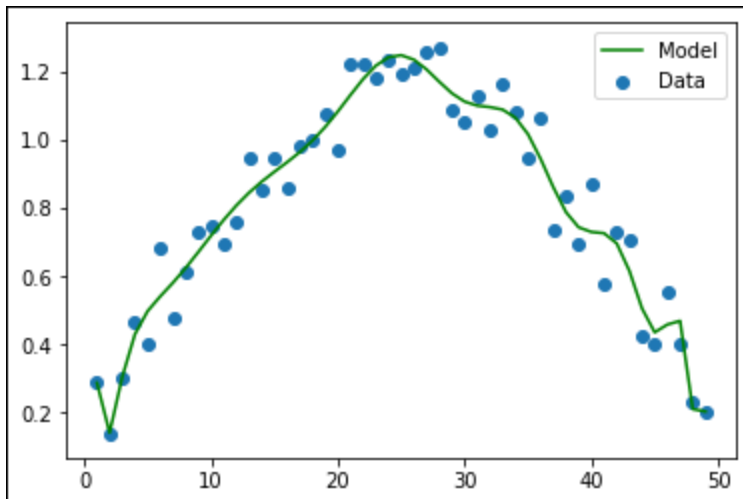


FIGURE 3-2: Overfitting causes the model to follow the data too closely.



WARNING The model might actually look good in this case. However, the brittleness of this model becomes apparent when you change the randomness of `vary`, as shown in the following code, to demonstrate the introduction of new data.

```
np.random.seed(22)
```

```
vary = (np.random.random(len(x))) / 3
```

```
y = np.sin(x * np.pi / 50.) + vary
```



```
plt.scatter(x, y)
```

```
plt.plot(x, p(x), 'g-')
```

```
plt.legend(['Model', 'Data'])
```

```
plt.show()
```

The new output, shown in [Figure 3-3](#), introduces all sorts of possible prediction errors. The problem with overfitting becomes evident in this case.

Plotting learning curves for insights

This chapter contains a number of examples of using algorithms and plots to see where problems may lie in data. Of course, nothing is guaranteed to work all the time, so you need to apply the correct algorithm to gain the insights you need. The examples in the previous sections demonstrate that you also need to experiment with the configuration of the algorithm — that is, experiment with how it performs its work. The following code shows what happens when you tune the degrees of polynomial fitting:

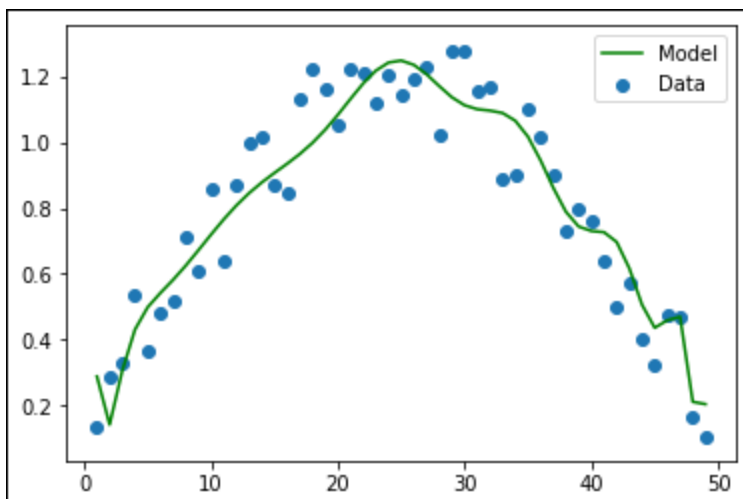


FIGURE 3-3: Applying the model to slightly different data shows the problem with overfitting in more detail.

```
np.random.seed(51)
```

```
x = np.array(range(1, 50))
```

```
vary = (np.random.random(len(x))) / 3
```

```
y = np.sin(x * np.pi / 50.) + vary
```

```
plt.scatter(x, y)
```

```
z = np.polyfit(x, y, 2)
```

```
p = np.poly1d(z)
```

```
plt.plot(x, p(x), 'g-')
```

```
np.random.seed(22)
```

```
vary = (np.random.random(len(x))) / 3
```

```
y = np.sin(x * np.pi / 50.) + vary
```

```
plt.scatter(x, y)
```

```
plt.legend(['Model', 'Data 1', 'Data 2'])
```

```
plt.show()
```



REMEMBER The output shown in [Figure 3-4](#) demonstrates that the model runs through the middle of both datasets with enough accuracy for you to perform predictions with some level of accuracy. However, the more important point is that this example also demonstrates the need to experiment with the algorithms at your disposal and the arguments they provide to create a useful model.

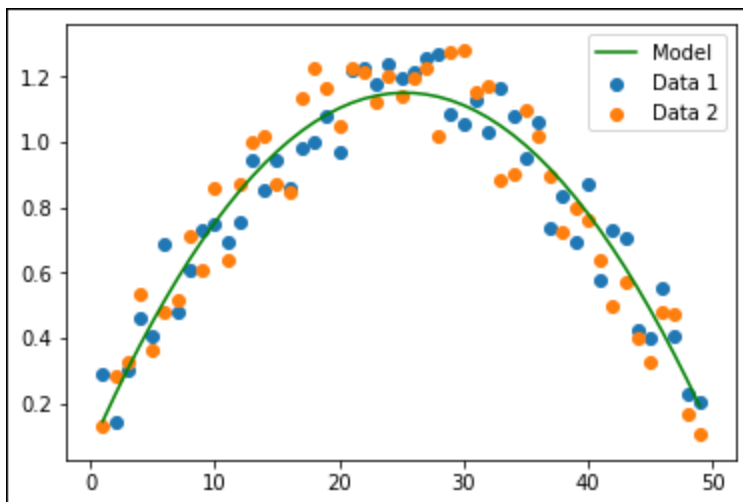


FIGURE 3-4: Using the correct degrees of polynomial fitting makes a big difference.

Determining the Sources of Overfitting and Underfitting

Merely knowing the causes of overfitting and underfitting isn't sufficient because the causes have a source that you must deal with in order to obtain the desired algorithmic output. For example, when you experience bias or variance, you need to look at the formulation of your model. The following sections help you understand the sources of overfitting and underfitting better.

NO SILVER BULLET IN ANALYSIS

You can see thousands of message threads on various forums online where someone is looking to solve a particular data problem precisely

and completely. However, in reviewing the material in this book, you often encounter terms like *confidence level* (how well an algorithm can predict an outcome) that would seem to say that you can only partly achieve your goal of perfection. This chapter considers the matter of overfitting and underfitting, but it comes with a caveat: You can't really achieve perfection in this area because the data will change over time and the methods used to model data aren't perfect. What you really end up with is a solution that works within a particular margin of error, not perfection. Driving yourself crazy trying to find perfection will only frustrate you and keep your project from completing on time. So, part of the hidden fix for the issues surrounding data analysis is deciding when the solution works well enough to meet your needs.

Understanding bias and variance

If your chosen learning algorithm can't learn properly from data and is not performing well, the cause is bias or variance in its estimates.

- **Bias:** Given the simplicity of a formulation, your algorithm tends to overestimate or underestimate the real rules behind the data and is systematically wrong in certain situations. Simple algorithms have high bias; having few internal parameters, they tend to represent only simple formulations well.
- **Variance:** Given the complexity of a formulation, your algorithm tends to learn too much information from the data and detects rules that don't exist, which causes its predictions to be erratic when faced with new data. You can think of variance as a problem connected to memorization. Complex algorithms can memorize data features thanks to the algorithm's high number of internal parameters. However, memorization doesn't imply any understanding about the rules.

Bias and variance depend on the complexity of the formulation at the core of the learning algorithm with respect to the complexity of the formulation that is presumed to have generated the data you are observing. However, when you consider a specific problem using the available data rules, you're better off having high bias or variance when

- **You have few observations:** Simpler algorithms perform better, no matter what the unknown function is. Complex algorithms tend to learn too much from data, causing inaccurate estimates.
- **You have many observations:** Complex algorithms always reduce variance. The reduction occurs because even complex algorithms can't learn all that much from data, so they learn just the rules, not any erratic noise.
- **You have many variables:** Provided that you also have many observations, simpler algorithms tend to find a way to approximate even complex hidden functions.

Having insufficient data

If you only have one data point, predicting the next data point is impossible (unless you're incredibly lucky). When you have two data points, predictions work only when the next data point happens to be in a straight line with the other two. Breaking the problem down this way makes it obvious that more data is better. Each additional data point you add can increase the accuracy of the prediction because you provide more data that the algorithm uses to learn.

Insufficient data tends to make your model underfit. The effect of insufficient data is that the model you create is inaccurate because it hasn't learned enough about the data. Without enough data points, you can't use something like least squares to create a model that accurately reflects the trend of the data. (Book 3, [Chapter 1](#) describes how linear regression uses the least squares method.)

Being fooled by data leakage

Data leakage is defined as the creation of unexpected additional information in your data (most especially training data, when you split your dataset into training and testing sets), allowing a model or machine learning algorithm to make unrealistically good predictions. Normally, this additional data isn't available to the model in the real world, so introducing it into the training process causes the learning

process to fail; the model learns incorrectly. Here are some common causes of data leakage:

- **Preprocessing:** The data is preprocessed in such a manner that it becomes skewed in an unrealistic way. For example, if you apply transformations to your training set, but also include the test set as part of the transformation, data will leak from the test set to the training set. Any transformations you apply should appear as part of the training set alone.
- **Duplicates:** The existence of the same data point, even from different observations, in both the training set and the test set could cause the model to fixate on that data point. Removing duplicates so that the training set and test set contain unique data points is essential.
- **Temporal information:** Data is especially prone to leakage when it relies on time because the variation of data over time tends to follow patterns that may not reflect reality in general. The best way to avoid this problem is to ensure that the training set and the test set contain a mix of data points across the entire time frame used for data collection.

Guessing the Right Features

Ensuring that you have the right *features*, which means the properties used to perform an analysis, is essential. The features determine what the algorithm considers when performing its analysis. Without the right information, the algorithm can't perform its task. The following sections discuss how to obtain and use the right features in your analysis.

Selecting variables like a pro

Selecting the right variables can improve the learning process by reducing the amount of noise (useless information) that can influence the learner's estimates. Variable selection, therefore, can effectively reduce the variance of predictions. In order to involve just the useful variables in training and leave out the redundant ones, you can use these techniques:

- **Univariate approach:** Select the variables most related to the target outcome.
- **Greedy or backward approach:** Keep only the variables that you can remove from the learning process without damaging its performance.

This example relies on the Boston dataset. The following code imports the dataset so that it's available for use:

```
from sklearn.datasets import load_boston
```

```
from sklearn.preprocessing import scale
```

```
from sklearn.metrics import mean_squared_error
```

```
boston = load_boston()
```

```
X = scale(boston.data)
```

```
y = boston.target
```

Selecting by univariate measures

If you decide to select a variable by its level of association with its target, the class `SelectPercentile` provides an automatic procedure for keeping only a certain percentage of the best, associated features. The available metrics for association are

- `f_regression`: Used only for numeric targets and based on linear regression performance
- `f_classif`: Used only for categorical targets and based on the Analysis of Variance (ANOVA) statistical test
- `chi2`: Performs the chi-square statistic for categorical targets, which is less sensitive to the nonlinear relationship between the predictive variable and its target

**TIP**

When evaluating candidates for a classification problem, `f_classif` and `chi2` tend to provide the same set of top variables. It's still a good practice to test the selections from both the association metrics.

Apart from applying a direct selection of the top percentile associations, `SelectPercentile` can also rank the best variables to make it easier to decide at what percentile to exclude a feature from participating in the learning process. The class `SelectKBest` is analogous in its functionality, but it selects the top `k` variables, where `k` is a number, not a percentile.

```
from sklearn.feature_selection import SelectPercentile

from sklearn.feature_selection import f_regression

Selector_f = SelectPercentile(f_regression, percentile=25)

Selector_f.fit(X, y)

for n,s in zip(boston.feature_names,Selector_f.scores_):

    print('F-score: %3.2f\t for feature %s ' % (s,n))
```

After a few iterations, the code prints the following results (your precise scores may differ from those shown):

```
F-score: 88.15      for feature CRIM
```

```
F-score: 75.26      for feature ZN
```

```
F-score: 153.95     for feature INDUS
```

```
F-score: 15.97      for feature CHAS
```


F-score: 112.59 for feature NOX

F-score: 471.85 for feature RM

F-score: 83.48 for feature AGE

F-score: 33.58 for feature DIS

F-score: 85.91 for feature RAD

F-score: 141.76 for feature TAX

F-score: 175.11 for feature PTRATIO

F-score: 63.05 for feature B

F-score: 601.62 for feature LSTAT

Using the level of association output (higher values signal more association of a feature with the target variable) helps you to choose the most important variables for your model, but you should watch out for these possible problems:

- Some variables with high association could also be highly correlated, introducing duplicated information, which acts as noise in the learning process.
- Some variables may be penalized, especially binary ones (variables indicating a status or characteristic using the value 1 when it is present, 0 when it is not). For example, notice that the output shows the binary variable `CHAS` as the least associated with the target variable (but you know from previous examples that it's influential during the cross-validation phase).



TIP

The univariate selection process can give you a real advantage when you have a huge number of variables to select from and all other methods turn computationally infeasible. The best procedure is to reduce the value of `SelectPercentile` by half or more of the avail-

able variables, reduce the number of variables to a manageable number, and consequently allow the use of a more sophisticated and more precise method, such as a greedy selection.

Using a greedy search

When using a univariate selection, you have to decide for yourself how many variables to keep: Greedy selection automatically reduces the number of features involved in a learning model on the basis of their effective contribution to the performance measured by the error measure.

The Boston dataset contains more than 500 observations and 13 features. The target is a price measure, so you decide to use linear regression to perform your analysis and to optimize the result using the mean squared error. The objective is to ensure that a linear regression is a good model for analyzing the Boston dataset and to quantify how good the analysis result is using the mean squared error (which lets you compare it with alternative models).

```
from sklearn.linear_model import LinearRegression
```

```
regression = LinearRegression()
```

```
regression.fit(X,y)
```

```
print('Mean squared error: %.2f' % mean_squared_error(
```

```
    y_true=y, y_pred=regression.predict(X)))
```

The output from this code is

```
Mean squared error: 21.89
```

The `RFECV` class, after fitting the data, can do the following:

- Give information about the number of useful features
- Point these features out to you
- Automatically transform the `X` data into a reduced variable set

as shown in the following example:

```
from sklearn.feature_selection import RFECV
```

```
selector = RFECV(estimator=regression,
```

```
cv=10,
```

```
scoring='neg_mean_squared_error')
```

```
selector.fit(X, y)
```

```
print("Optimal number of features : %d"
```

```
% selector.n_features_)
```

The example outputs an optimal number of features for the problem:

```
Optimal number of features: 6
```

Obtaining an index to the optimum variable set is possible by calling the attribute `support_` from the `RFECV` class after you fit it:

```
print(boston.feature_names[selector.support_])
```

The command will print the list containing the features:

```
['CHAS' 'NOX' 'RM' 'DIS' 'PTRATIO' 'LSTAT']
```

Notice that `CHAS` is now included among the most predictive features, which contrasts with the result from the univariate search in the previous section. The `RFECV` method can detect whether a variable is important, no matter whether it is binary, categorical, or numeric, be-

cause it directly evaluates the role played by the feature in the prediction.



REMEMBER The **RFECV** method is certainly more efficient, when compared to the univariate approach, because it considers highly correlated features and is tuned to optimize the evaluation measure (which usually is not Chi-square or F-score). Being a greedy process, it's somehow computationally demanding and may only approximate the best set of predictors.



TIP As **RFECV** learns the best set of variables from data, the selection may overfit, which is what happens with all other machine learning algorithms. Trying **RFECV** on different samples of the training data can confirm the best variables to use.

Using nonlinear transformations

Linear models, such as linear and logistic regression, are actually linear combinations that sum your features (weighted by learned coefficients) and provide a simple but effective model. In most situations, they offer a good approximation of the complex reality they represent. Even though they're characterized by a high bias, using a large number of observations can improve their coefficients and make them more competitive when compared to complex algorithms.

However, they can perform better when solving certain problems if you pre-analyze the data using the Exploratory Data Analysis (EDA) approach. After performing the analysis, you can transform and enrich the existing features by:

- Linearizing the relationships between features and the target variable using transformations that increase their correlation and make their point cloud in the scatterplot more similar to a line (see <https://whatis.techtarget.com/definition/point-cloud> for details on how point clouds work).
- Making variables interact by multiplying them so that you can better represent their conjoint behavior.
- Expanding the existing variables using the polynomial expansion in order to represent relationships more realistically (such as ideal point curves, when there is a peak in the variable representing a maximum, akin to a parabola).

Doing variable transformations

An example is the best way to explain the kind of transformations you can successfully apply to data to improve a linear model. The example in this section, and the “[Regularizing linear models](#)” section that follows, relies on the Boston dataset. The problem relies on regression, and the data originally has ten variables to explain the different housing prices in Boston during the 1970s. The dataset also has implicit ordering. Fortunately, order doesn’t influence most algorithms because they learn the data as a whole. When an algorithm learns in a progressive manner, ordering can interfere with effective model building. By using `seed` (to fix a preordained sequence of random numbers) and `shuffle` from the `random` package (to shuffle the index), you can reindex the dataset.

```
import random
```

```
from random import shuffle
```

```
random.seed(0) # Creates a replicable shuffling
```

```
new_index = list(range(boston.data.shape[0]))
```

```
shuffle(new_index) # shuffling the index
```

```
X, y = boston.data[new_index], boston.target[new_index]
```

```
print(X.shape, y.shape, boston.feature_names)
```

In the code, `random.seed(0)` creates a replicable shuffling operation, and `shuffle(new_index)` creates the new shuffled index used to re-order the data. After that, the code prints the `X` and `y` shapes as well as the list of dataset variable names:

```
(506, 13) (506,) ['CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM'
```

```
'AGE' 'DIS' 'RAD' 'TAX' 'PTRATIO' 'B' 'LSTAT']
```

**TIP**

You can find out more detail about the meaning of the variables present in the Boston dataset by issuing the following command: `print(boston.DESCR)`. You see the output of this command in the downloadable source code.

Converting the array of predictors and the target variable into a `pandas DataFrame` helps support the series of explorations and operations on data. Moreover, although Scikit-learn requires an `ndarray` as input, it will also accept `DataFrame` objects:

```
import pandas as pd
```

```
df = pd.DataFrame(X, columns=boston.feature_names)
```

```
df['target'] = y
```

The best way to spot possible transformations is by graphical exploration, and using a scatterplot can tell you a lot about two variables. You need to make the relationship between the predictors and the tar-

get outcome as linear as possible, so you should try various combinations, such as the following:

```
ax = df.plot(kind='scatter', x='LSTAT', y='target', c='b')
```

In [Figure 3-5](#), you see a representation of the resulting scatterplot. Notice that you can approximate the cloud of points by using a curved line rather than a straight line. In particular, when LSTAT is around 5, the target seems to vary between values of 20 to 50. As LSTAT increases, the target decreases to 10, reducing the variation.

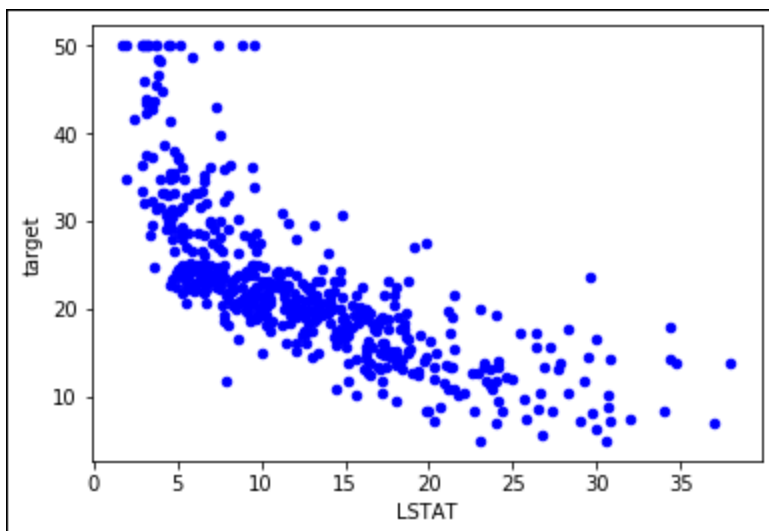


FIGURE 3-5: Nonlinear relationship between variable LSTAT and target prices.

Logarithmic transformation can help in such conditions. However, your values, such as percentages, should range from zero to one, as demonstrated in this example. In other cases, other useful transformations for your `x` variable could include `x**2`, `x**3`, `1/x`, `1/x**2`, `1/x**3`, and `sqrt(x)`. The key is to try them and test the result. As for testing, you can use the following script as an example:

```
single_variable = df['LSTAT'].values.reshape(-1, 1)
```

```
F, pval = f_regression(single_variable, y)
```

```
print('F score for the original feature %.1f' % F)
```

```
F, pval = f_regression(np.log(single_variable),y)
```

```
print('F score for the transformed feature %.1f' % F)
```

The code prints the **F score**, a measure to evaluate how a feature is predictive in a machine learning problem, both the original and the transformed feature. The score for the transformed feature is a great improvement over the untransformed one.

```
F score for the original feature 601.6
```

```
F score for the transformed feature 1000.2
```

The **F score** is useful for variable selection. You can also use it to assess the usefulness of a transformation because both **f_regression** and **f_classif** are themselves based on linear models, and are therefore sensitive to every effective transformation used to make variable relationships more linear.

Creating interactions between variables

In a linear combination, the model reacts to how a variable changes in an independent way with respect to changes in the other variables. In statistics, this kind of model is a *main effects model*.



REMEMBER The Naïve Bayes classifier makes a similar assumption for probabilities; also, it works well with complex text problems.

Even though machine learning works by using approximations, and a set of independent variables can make your predictions work well in most situations, sometimes you may miss an important part of the picture. You can easily catch this problem by depicting the variation in

your target associated with the conjoint variation of two or more variables in two simple and straightforward ways:

- **Existing domain knowledge of the problem:** For instance, in the car market, having a noisy engine is a nuisance in a family car but considered a plus for sports cars. (Car aficionados want to hear that you have an ultra-cool and expensive car.) By knowing a consumer preference, you can model a noise level variable and a car-type variable together to obtain exact predictions using a predictive analytic model that guesses the car's value based on its features.
- **Testing combinations of different variables:** By performing group tests, you can see the effect that certain variables have on your target variable. Therefore, even without knowing about noisy engines and sports cars, you could have caught a different average of preference level when analyzing your dataset split by type of cars and noise level.

The following example shows how to test and detect interactions in the Boston dataset. The first task is to load a few helper classes, as shown here:

```
from sklearn.model_selection import cross_val_score, KFold

regression = LinearRegression(normalize=True)

crossvalidation = KFold(n_splits=10, shuffle=True,

                        random_state=1)
```

The code reinitializes the `pandas DataFrame` using only the predictor variables. A `for` loop matches the different predictors and creates a new variable containing each interaction. The mathematical formulation of an interaction is simply a multiplication.

```
df = pd.DataFrame(X, columns=boston.feature_names)

baseline = np.mean(cross_val_score(regression, df, y,
```

```
        scoring='r2',

        cv=crossvalidation))

interactions = list()

for var_A in boston.feature_names:

    for var_B in boston.feature_names:

        if var_A > var_B:

            df['interaction'] = df[var_A] * df[var_B]

            cv = cross_val_score(regression, df, y,

                                scoring='r2',

                                cv=crossvalidation)

            score = round(np.mean(cv), 3)

            if score > baseline:

                interactions.append((var_A, var_B, score))

print('Baseline R2: %.3f' % baseline)

print('Top 10 interactions: %s' % sorted(interactions,

                                         key=lambda x :x[2],

                                         reverse=True)[:10])
```

The code starts by printing the baseline R^2 score for the regression; then it reports the top ten interactions whose addition to the mode increases the score:

```
Baseline R2: 0.716
```

```
Top 10 interactions: [('RM', 'LSTAT', 0.79), ('TAX', 'RM',
0.782), ('RM', 'RAD', 0.778), ('RM', 'PTRATIO', 0.766),
('RM', 'INDUS', 0.76), ('RM', 'NOX', 0.747), ('RM',
'AGE', 0.742), ('RM', 'B', 0.738), ('RM', 'DIS', 0.736),
('ZN', 'RM', 0.73)]
```

The code tests the specific addition of each interaction to the model using a 10 folds cross-validation. The code records the change in the R^2 measure into a stack (a simple list) that an application can order and explore later.

The baseline score is 0.699, so a reported improvement of the stack of interactions to 0.782 looks quite impressive. It's important to know how this improvement is made possible. The two variables involved are RM (the average number of rooms) and LSTAT (the percentage of lower-status population). A plot will disclose the case about these two variables:

```
colors = ['b' if v > np.mean(y) else 'r' for v in y]
```

```
scatter = df.plot(kind='scatter', x='RM', y='LSTAT',
```

```
c=colors)
```

The scatterplot in [Figure 3-6](#) clarifies the improvement. In a portion of houses at the center of the plot, you need to know both LSTAT and RM to correctly separate the high-value houses from the low-value houses; therefore, an interaction is indispensable in this case.

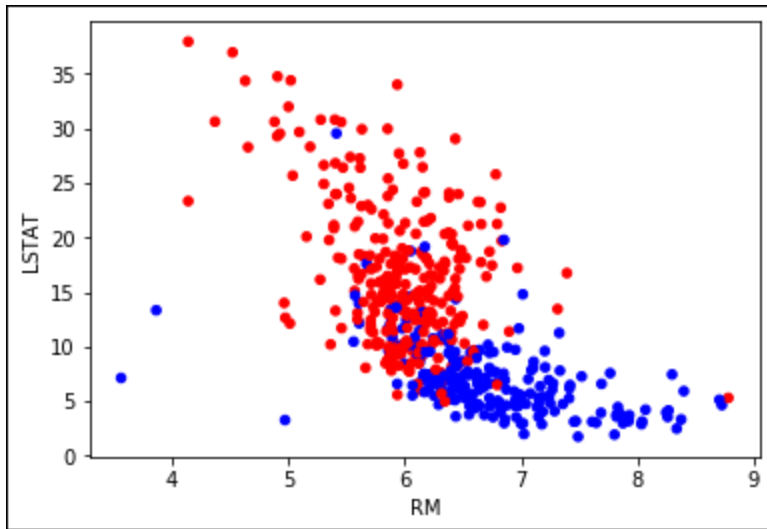


FIGURE 3-6: Combined variables LSTAT and RM help to separate high from low prices.

Adding interactions and transformed variables leads to an extended linear regression model, a polynomial regression. Data scientists rely on testing and experimenting to validate an approach to solving a problem, so the following code slightly modifies the previous code to redefine the set of predictors using interactions and quadratic terms by squaring the variables:

```
polyX = pd.DataFrame(X, columns=boston.feature_names)
```

```
cv = cross_val_score(regression, polyX, y,
```

```
scoring='neg_mean_squared_error',
```

```
cv=crossvalidation)
```

```
baseline = np.mean(cv)
```

```
improvements = [baseline]
```

```
for var_A in boston.feature_names:
```

```
polyX[var_A+'^2'] = polyX[var_A]**2
```

```
cv = cross_val_score(regression, polyX, y,
```

```
        scoring='neg_mean_squared_error',  
  
        cv=crossvalidation)  
  
    improvements.append(np.mean(cv))  
  
    for var_B in boston.feature_names:  
  
        if var_A > var_B:  
  
            poly_var = var_A + '*' + var_B  
  
            polyX[poly_var] = polyX[var_A] * polyX[var_B]  
  
            cv = cross_val_score(regression, polyX, y,  
  
                                scoring='neg_mean_squared_error',  
  
                                cv=crossvalidation)  
  
            improvements.append(np.mean(cv))
```

This bit of code plots the results for you:

```
plt.figure()  
  
plt.plot(range(0,92),np.abs(improvements),'-')  
  
plt.xlabel('Added polynomial features')  
  
plt.ylabel('Mean squared error')  
  
plt.show()
```

To track improvements as the code adds new, complex terms, the example places values in the `improvements` list. [Figure 3-7](#) shows a graph of the results. The graph demonstrates that some additions are

great because they decrease the squared error, and other additions are terrible because they increase the error instead.

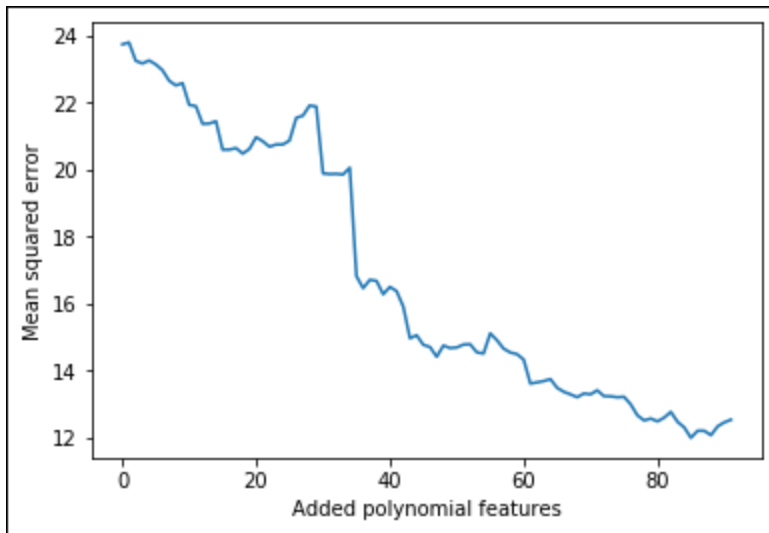


FIGURE 3-7: Adding polynomial features increases the predictive power.

Of course, instead of unconditionally adding all the generated variables, you could perform an ongoing test before deciding to add a quadratic term or an interaction, checking by cross-validation to see whether each addition is really useful for your predictive purposes. This example is a good foundation for checking other ways of controlling the existing complexity of your datasets or the complexity that you have to induce with transformation and feature creation in the course of data-exploration efforts. Before moving on, you check both the shape of the actual dataset and its cross-validated mean squared error:

```
print('New shape of X:', np.shape(polyX))
```

```
crossvalidation = KFold(n_splits=10, shuffle=True,
```

```
random_state=1)
```

```
cv = cross_val_score(regression, polyX, y,
```

```
scoring='neg_mean_squared_error',
```

```
cv=crossvalidation)
```

```
print('Mean squared error: %.3f' % abs(np.mean(cv)))
```

Even though the mean squared error is good, the ratio between 506 observations and 104 features isn't all that good because the number of observations may not be enough for a correct estimate of the coefficients.

```
New shape of X: (506, 104)
```

```
Mean squared error: 12.514
```

**TIP**

As a rule of thumb, divide the number of observations by the number of coefficients. The code should have at least 10 to 20 observations for every coefficient you want to estimate in linear models. However, experience shows that having at least 30 of them is better.

Regularizing linear models

Linear models have a high bias, but as you add more features, more interactions, and more transformations, they start gaining adaptability to the data characteristics and memorizing power for data noise, thus increasing the variance of their estimates. Trading higher variance for less bias isn't always the best choice, but, as mentioned earlier, sometimes it's the only way to increase the predictive power of linear algorithms.

You can introduce L1 and L2 regularization as a way to control the trade-off between bias and variance in favor of an increased generalization capability of the model. When you introduce one of the regularizations, an additive function that depends on the complexity of the linear model penalizes the optimized cost function. In linear regres-

sion, the cost function is the squared error of the predictions, and the cost function is penalized using a summation of the coefficients of the predictor variables.

If the model is complex but the predictive gain is little, the penalization forces the optimization procedure to remove the useless variables, or to reduce their impact on the estimate. The regularization also acts on highly correlated features — attenuating or excluding their contribution, thus stabilizing the results and reducing the consequent variance of the estimates:

- **L1 (also called Lasso):** Shrinks some coefficients to zero, making your coefficients sparse. It performs variable selection.
- **L2 (also called Ridge):** Reduces the coefficients of the most problematic features, making them smaller, but seldom equal to zero. All coefficients keep participating in the estimate, but many become small and irrelevant.



REMEMBER You can control the strength of the regularization using a hyperparameter, usually a coefficient itself, often called `alpha`. When `alpha` approaches 1.0, you have stronger regularization and a greater reduction of the coefficients. In some cases, the coefficients are reduced to zero. Don't confuse `alpha` with `C`, a parameter used by `LogisticRegression` and by support vector machines, because `C` is $1/\alpha$, so it can be greater than 1. Smaller `C` numbers actually correspond to more regularization, exactly the opposite of `alpha`.



TIP Regularization works because it is the sum of the coefficients of the predictor variables; therefore they need to be on the same scale or the regularization may find it difficult to converge, and variables with larger absolute coefficient values will greatly influence it, generating an ineffective regularization. It's good practice to standardize the

predictor values or bind them to a common min-max, such as the `[-1,+1]` range. The following sections demonstrate various methods of using both L1 and L2 regularization to achieve various effects.

Relying on Ridge regression (L2)

The first example uses the L2 type regularization, reducing the strength of the coefficients. The `Ridge` class implements L2 for linear regression. Its usage is simple; it presents just the parameter `alpha` to fix. `Ridge` also has another parameter, `normalize`, that automatically normalizes the inputted predictors to zero mean and unit variance.

```
from sklearn.model_selection import GridSearchCV

from sklearn.linear_model import Ridge

ridge = Ridge(normalize=True)

search_grid = {'alpha':np.logspace(-5,2,8)}

search = GridSearchCV(estimator=ridge,

                      param_grid=search_grid,

                      scoring='neg_mean_squared_error',

                      refit=True, cv=10, iid=False)

search.fit(polyX,y)

print('Best parameters: %s' % search.best_params_)

score = abs(search.best_score_)

print('CV MSE of best parameters: %.3f' % score)
```

After searching for the best `alpha` parameter, the resulting best model is

```
Best parameters: {'alpha': 0.001}
```

```
CV MSE of best parameters: 11.630
```

**TIP**

A good search space for the `alpha` value is in the range `np.logspace(-5, 2, 8)`. Of course, if the resulting optimum value is on one of the extremities of the tested range, you need to enlarge the range and retest.



REMEMBER The `polyX` and `y` variables used for the examples in this section and the sections that follow are created as part of the example in the “[Creating interactions between variables](#)” section, earlier in this chapter. If you haven't worked through that section, the examples in this section will fail to work properly.

Using the Lasso (L1)

The second example uses the L1 regularization, the `Lasso` class, whose principal characteristic is to reduce the effect of less useful coefficients down toward zero. This action enforces sparsity in the coefficients, with just a few having values above zero. The class uses the same parameters of the `Ridge` class that are demonstrated in the previous section.

```
from sklearn.linear_model import Lasso
```

```
lasso = Lasso(normalize=True, tol=0.05, selection='random')
```

```
search_grid = {'alpha':np.logspace(-2,3,8)}

search = GridSearchCV(estimator=lasso,

                      param_grid=search_grid,

                      scoring='neg_mean_squared_error',

                      refit=True, cv=10, iid=False)

search.fit(polyX,y)

print('Best parameters: %s' % search.best_params_)

score = abs(search.best_score_)

print('CV MSE of best parameters: %.3f' % score)
```

In setting the `Lasso`, the code uses a less sensitive algorithm (`tol=0.05`) and a random approach for its optimization (`selection='random'`). The resulting mean squared error obtained is higher than it is using the L2 regularization:

```
Best parameters: {'alpha': 0.01}
```

```
CV MSE of best parameters: 20.406
```

Leveraging regularization

Because you can indent the sparse coefficients resulting from a L1 regression as a feature selection procedure, you can effectively use the `Lasso` class for selecting the most important variables. By tuning the `alpha` parameter, you can select a greater or lesser number of variables. In this case, the code sets the `alpha` parameter to 0.01, obtaining a much simplified solution as a result:

```
lasso = Lasso(normalize=True, alpha=0.01)
```

```
lasso.fit(polyX,y)
```

```
print(polyX.columns[np.abs(lasso.coef_)>0.0001].values)
```

The simplified solution is made of a handful of interactions:

```
['CRIM*CHAS' 'ZN*CRIM' 'ZN*CHAS' 'INDUS*DIS' 'CHAS*B'
```

```
'NOX^2' 'NOX*DIS' 'RM^2' 'RM*CRIM' 'RM*NOX' 'RM*PTRATIO'
```

```
'RM*B' 'RM*LSTAT' 'RAD*B' 'TAX*DIS' 'PTRATIO*NOX'
```

```
'LSTAT^2']
```



TIP

You can apply L1-based variable selection automatically to both regression and classification using the `RandomizedLasso` and `RandomizedLogisticRegression` classes. Both classes create a series of randomized L1 regularized models. The code keeps track of the resulting coefficients. At the end of the process, the application keeps any coefficients that the class didn't reduce to zero because they're considered important. You can train the two classes using the `fit` method, but they don't have a `predict` method; instead, they have just a `transform` method that effectively reduces your dataset, as is true of most classes in the `sklearn.preprocessing` module.

Combining L1 & L2: Elasticnet

L2 regularization reduces the impact of correlated features, whereas L1 regularization tends to select them. A good strategy is to mix them using a weighted sum by using the `ElasticNet` class. You control both L1 and L2 effects by using the same alpha parameter, but you can de-

cide the L1 effect's share by using the `l1_ratio` parameter. Clearly, if `l1_ratio` is 0, you have a `Ridge` regression; on the other hand, when `l1_ratio` is 1, you have a `Lasso`.

```
from sklearn.linear_model import ElasticNet

elastic = ElasticNet(normalize=True, selection='random')

search_grid = {'alpha':np.logspace(-4,3,8),

               'l1_ratio': [0.10 ,0.25, 0.5, 0.75]}

search = GridSearchCV(estimator=elastic,

                      param_grid=search_grid,

                      scoring='neg_mean_squared_error',

                      refit=True, cv=10, iid=False)

search.fit(polyX,y)

print('Best parameters: %s' % search.best_params_)

score = abs(search.best_score_)

print('CV MSE of best parameters: %.3f' % score)
```

After a while, you get a result that's quite comparable to L1's:

```
Best parameters: {'alpha': 0.0001, 'l1_ratio': 0.75}

CV MSE of best parameters: 12.581
```

Table of contents collapsed